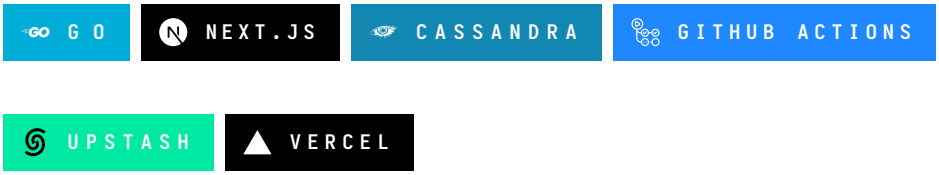
Serverless Esports Tracking & Notification System



> *A zero-maintenance, event-driven pipeline for real-time esports match tracking and pre-match notifications — no always-on servers required.*

What This Is

Most esports tracking tools rely on a persistent backend process: a server that sits idle 99% of the time, polling APIs and waiting. This project eliminates that entirely.

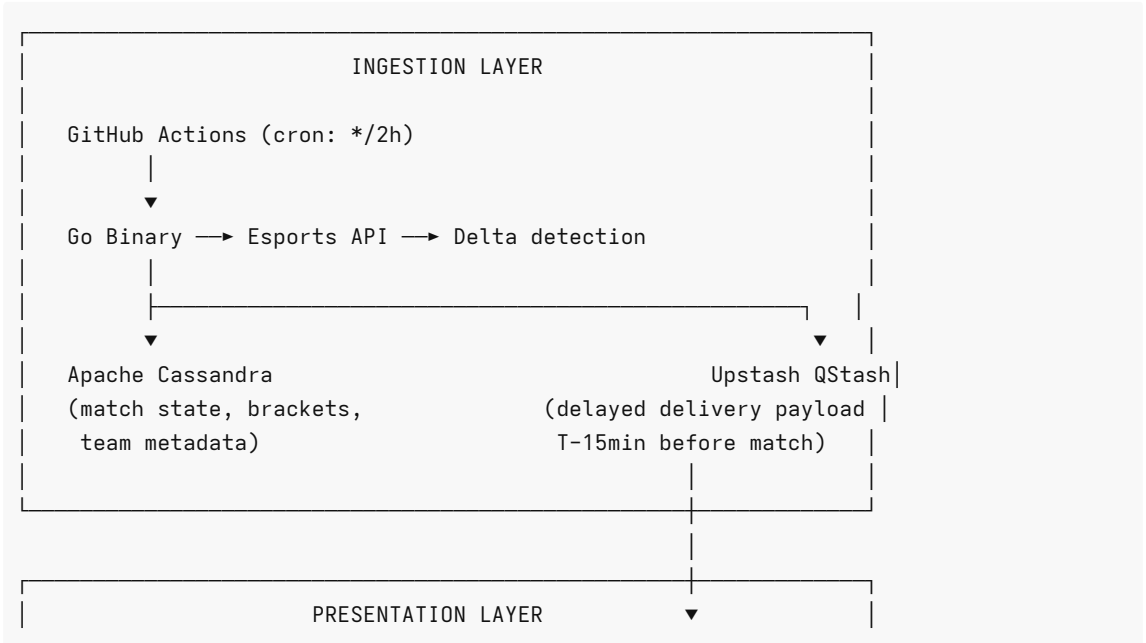
Instead, it uses a **hybrid serverless pipeline** — ephemeral compute triggered on a schedule, a wide-column store built for write-heavy workloads, and a serverless message broker to handle time-delayed notifications. Infrastructure cost at scale: effectively zero.

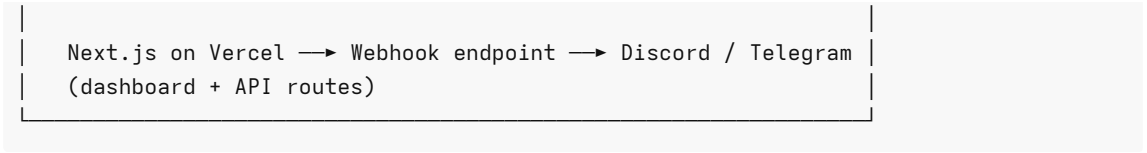
The system is broken into three strictly isolated concerns:

- **Ingestion** — fetch match data from third-party APIs
- **Persistence** — store and query it efficiently
- **Notification** — alert users at the right moment, before a match starts

---

Architecture





Tech Stack

Layer	Technology	Role
Ingestion	Go + GitHub Actions	Concurrent API polling on a cron schedule
Storage	Apache Cassandra (DataStax Astra DB)	Wide-column store for time-series match data
Messaging	Upstash QStash	Serverless broker for delayed webhook dispatch
Frontend	Next.js (Vercel)	Dashboard UI + edge API webhook receiver

---

Component Breakdown

1. Ingestion Engine — Go + GitHub Actions

The ingestion layer is a stateless Go binary executed by GitHub Actions on a defined cron schedule (default: every 2 hours).

Go was chosen deliberately — fast cold starts, minimal memory footprint, and goroutines that allow parallel fetching of multiple tournament brackets in a single run.

Key behaviors:

- Each pipeline run is fully independent. No shared state between executions.
- Goroutines fan out across tournament endpoints concurrently.
- Only delta changes are written to the database — no redundant upserts.
- On new match detection, a payload is pushed to QStash with a delivery delay (e.g., T-15 minutes before match start).

2. Persistence Layer — Apache Cassandra

Match states, tournament brackets, and team metadata are stored in Apache Cassandra, provisioned via DataStax Astra DB for serverless operation.

Cassandra's wide-column model is a deliberate choice for this workload. Esports match data is write-heavy, time-ordered, and queried by tournament or team — exactly the access pattern Cassandra's partition model is designed for. It handles high write throughput without the lock contention of a traditional RDBMS.

3. Messaging Layer — Upstash QStash

Sending a notification "15 minutes before match start" is a scheduling problem, not just a messaging one. QStash solves this without a persistent worker process.

When the Go agent detects a new match:

1. It constructs a notification payload.
2. It publishes to QStash with a calculated delivery delay.
3. QStash holds the message and delivers it to the Next.js webhook endpoint at the configured time.

No polling. No cron job for notifications. No always-on worker.

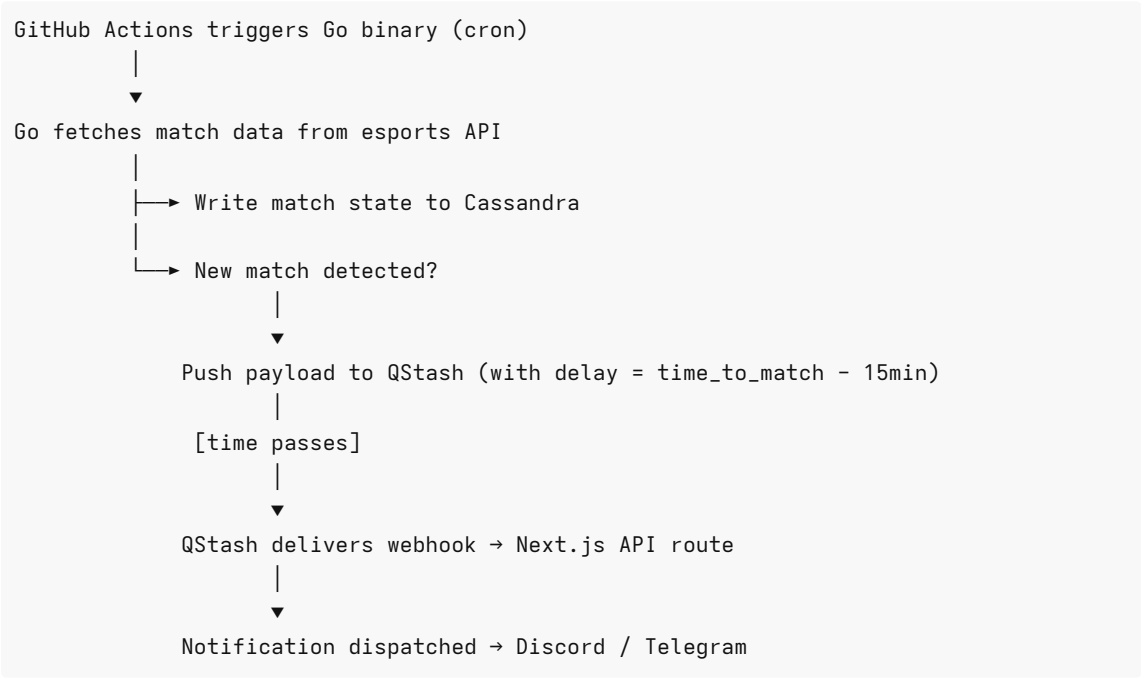
4. Presentation Layer — Next.js on Vercel

The Next.js app serves two purposes:

- **Dashboard** — displays upcoming matches, tournament brackets, and team data pulled from Cassandra.
- **Webhook receiver** — edge API routes accept incoming QStash payloads and route final alerts to Discord or Telegram.

Hosting on Vercel keeps the frontend serverless as well — edge functions handle webhook traffic without a standing server.

Data Flow



Why Serverless for This?

Concern	Traditional Approach	This System
Match polling	Always-on server process	GitHub Actions cron — runs, exits

Notification scheduling	Cron worker or timer queue	QStash delayed delivery
API serving	Managed server	Vercel edge functions
Database	Self-hosted or provisioned	DataStax Astra DB (serverless Cassandra)
Total idle cost	Linear with uptime	**Effectively $0**

The tradeoff is cold start latency on the Go binary (negligible for a polling job) and eventual consistency on Cassandra reads (acceptable for match data that changes on the order of minutes, not milliseconds).

Local Development

```
# Clone the repo
git clone https://github.com/your-username/esports-tracker.git
cd esports-tracker

# Run the Go ingestion binary locally
cd ingestion
go run main.go

# Run the Next.js frontend
cd ../frontend
npm install
npm run dev
```

Environment variables required:

```
# Esports API
ESPORTS_API_KEY=

# Cassandra / Astra DB
ASTRA_DB_TOKEN=
ASTRA_DB_ENDPOINT=
ASTRA_KEYSPACE=

# Upstash QStash
QSTASH_TOKEN=
QSTASH_CURRENT_SIGNING_KEY=
QSTASH_NEXT_SIGNING_KEY=

# Notification targets
DISCORD_WEBHOOK_URL=
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=
```

GitHub Actions — Cron Configuration

```yaml
# .github/workflows/ingest.yml
on:
  schedule:
    - cron: "0 */2 * * *" # Every 2 hours
  workflow_dispatch: # Manual trigger for testing
```

Project Structure

```
esports-tracker/
├── ingestion/              # Go binary — API polling + Cassandra writes + QStash
dispatch
│   ├── main.go
│   ├── api/
│   ├── db/
│   └── broker/
├── frontend/               # Next.js app — dashboard + webhook receiver
│   ├── app/
│   │   ├── page.tsx
│   │   └── api/
│   │       └── webhook/
│   └── components/
└── .github/
    └── workflows/
        └── ingest.yml
```

License

MIT

▨ Project Roadmap

- [x] **Phase 1**: Ingestion Pipelines & Frontend Scaffold.
- [ ] **Phase 2**: Dedicated Monolith Backend Development.
- [ ] **Phase 3**: Microservices Integration.
- [ ] **Phase 4**: Full-Stack Polish.